PortfolioOptimisers.jl

Daniel Celis Garza[1]

[1]Independent researcher, Oxford, UK

July 13, 2026

Abstract

Portfolio optimisation is the science of either: *1)* Minimising risk whilst keeping returns to acceptable levels. *2)* Maximising returns whilst keeping risk to acceptable levels. To some definition of acceptable, and with any number of additional constraints available to the optimisation type. There exist myriad statistical, pre- and post-processing, optimisations, and constraints that allow one to explore an extensive landscape of "optimal" portfolios. PortfolioOptimisers.jl is an attempt at providing as many of these as possible under a single banner and making it accessible to all. We make extensive use of Julia's type system, module extensions, and multiple dispatch to simplify development and maintenance, while keeping robustness, testability, and usability high.

Keywords. Portfolio Optimisation · Quantitative Investment · Conic Optimisation · Parameter Estimation

1 Introduction

The field of portfolio optimisation was introduced by Harry Markowitz's seminal 1952 paper "Modern portfolio theory" [1]. However, the field has grown significantly since then, with many new techniques and methods being developed. The original Markowitz model is highly sensitive to estimation errors in the input parameters, particularly the expected returns. After all, it attempts to summarise the entire distribution of returns in highly compressed summary statistics, both of which can be sensitive to outliers and atypical conditions for the lookback period. Many have studied and addressed the strengths and weaknesses of the Markowitz model [2], [3], [4].

Unfortunately, most implementations live in disparate, unconnected, often propriatary, bespoke codebases, which limit their applicability. It is only recently that various advanced yet very usable libraries have been published [5], [6], [7]. However, each has its own strengths, weaknesses, scope, and ideosyncrasies. That is no different from PortfolioOptimisers.jl, but it is our hope that this library serves as a unifying framework for the various techniques and methods available in the field, while also eventually providing a simple and intuitive interface for users, with advanced features for experts.

There also exist myriad optimisation methods, pre-filtering, distribution and moment estimators, machine learning techniques, validation, and parameter tuning methods that can all be used together to improve the out-of sample performance of a portfolio. To this day, only [7] has succeeded in providing a unified framework to do this. PortfolioOptimisers.jl provides an alternative that is not tied to the scikit-learn [8] or cvxpy [9], [10] APIs and ecosystems, as well as providing different functionality and a different architectural philosphy.

2 Design and implementation

2.1 Basic usage

??

2.2 Design philosophy

PortfolioOptimisers.jl is implemented in Julia [11], using the JuMP.jl [12] mathematical optimisation embedded language, and leverages other well-established Julia libraries for much of its functionality. Aside from a strong commitment to FOSS principles, the library is designed atop four pillars:

1. Well-defined type hierachies. This lets us take advantage of generic fallbacks, and enables fearless extensibility, even at the user-level thanks to multiple dispatch.
2. Strongly typed, immutable structs. This lets the compiler agressively optimise code, provides a single source of truth, and enables construction-time data validation that will not expire.
3. Compositional design. This lets us build complex objects from simpler ones, enables code reuse, and aids in extensibility.
4. Defensive programming. This lets us catch errors as early as possible, and provides a clear and consistent interface to the user.

With the exception of covariance estimators, which are defined in StatsBase.jl, every type in the library is a subtype of one of three types:

1. `AbstractEstimator`: These contain all the parameters to estimate a particular quantity. They are consumed by functions that perform the estimation. Estimators are used at user-level, they can be used by the public API to return a result. Estimators can be nested within each other, and can be used to build more complex estimators.
2. `AbstractResult`: Many of the estimators return more than one quantity, these are returned in a result type.
3. `AbstractAlgorithm`: These modify the behaviour of estimators via multiple dispatch. They are not used standalone or directly by the public API, but as part of an estimator.

The strict adherence to a well-defined type hierarchy lets developers and users alike extend and modify the library's behaviour without modifying the source code.

The code is available under an MIT license via the Julia General registry. It follows common software development best practices:

1. An extensive and high coverage test suite.
2. Automated documentation builds with an introductory user guide and deep examples, as well as a completely documented public and private API.
3. Miscelaneous code quality checks.

All as part of a GitHub continuous integration pipeline. It is under active development, and receives regular updates and improvements.

3 Equations

The template uses `i-figured` for labeling equations. Equations will be numbered only if they are labelled. Here is an equation with a label:

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2} \tag{3.1}$$

We can reference it by `@eq:label` like this: (3.1), i.e., we need to prepend the label with `eq:`. The number of an equation is determined by the section it is in, i.e. the first digit is the section number and the second digit is the equation number within that section.

Here is an equation without a label:

$$\exp(x) = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

As we can see, it is not numbered.

4 Theorems

The template uses `great-theorems` for theorems. Here is an example of a theorem:

Theorem 4.1. (Example Theorem): *This is an example theorem.*

Proof. This is the proof of the example theorem. □

We also provide `definition`, `lemma`, `remark`, `example`, and `questions` among others. Here is an example of a definition:

Definition 4.2. (Example Definition): *This is an example definition.*

***Question* 4.3.** (Custom mathblock?): How do you define a custom mathblock?

Answer 4.4. *You can define a custom mathblock like this:*

```
#let answer = my-mathblock(
  blocktitle: "Answer",
  bodyfmt: text.with(style: "italic"),
)
```

Similar as for the equations, the numbering of the theorems is determined by the section they are in. We can reference theorems by `@label` like this: Theorem 4.1.

To get a bibliography, we also add a citation.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Bibliography

[1] H. Markowitz, "Modern portfolio theory," *Journal of Finance*, vol. 7, no. 11, pp. 77–91, 1952.

[2] D. Cajas, *ADVANCED PORTFOLIO OPTIMIZATION*. Springer, 2025.

[3] M. L. De Prado, *Advances in financial machine learning*. John Wiley & Sons, 2018.

[4] P. P. DANIEL, *Portfolio Optimization: Theory and Application*. CAMBRIDGE University Press, 2025.

[5] D. P. Palomar, "Portfolio Optimization." [Online]. Available: https://github.com/dppalomar

[6] D. Cajas, "Riskfolio-Lib (7.3)." [Online]. Available: https://github.com/dcajasn/Riskfolio-Lib

[7] H. Delatte, C. Nicolini, and M. Manzi, "skfolio." [Online]. Available: https://doi.org/10.5281/zenodo.16148630

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[9] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[10] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, "A rewriting system for convex optimization problems," *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

[11] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017, doi: 10.1137/141000671.

[12] M. Lubin, O. Dowson, J. Dias Garcia, J. Huchette, B. Legat, and J. P. Vielma, "JuMP 1.0: Recent improvements to a modeling language for mathematical optimization," *Mathematical Programming Computation*, vol. 15, pp. 581–589, 2023, doi: 10.1007/s12532-023-00239-3.

Appendix A

If you have appendices, you can add them after `#show: appendices`. The appendices are started with an empty heading `=` and will be numbered alphabetically. Any appendix can also have different subsections.

A.1 Appendix section

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.